```
# Step 1: Import libraries and load video

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

uploaded = files.upload()

video_path = list(uploaded.keys())[0]

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Video not found.")
else:
    print("Video loaded successfully.")
```

Choose Files 3987648-h…0_24fps.mp4

3987648-hd_1920_1080_24fps.mp4(video/mp4) - 11587836 bytes, last modified: 9/29/2026 - 100% done

```
Saving 3987648-hd_1920_1080_24fps.mp4 to 3987648-hd_1920_1080_24fps.mp4
Video loaded successfully.
```

```
# Step 2: Read consecutive frames and convert to grayscale

ret, frame1 = cap.read()
ret, frame2 = cap.read()

if ret:
    gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.imshow(gray1, cmap="gray")
    plt.title("Frame 1")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(gray2, cmap="gray")
    plt.title("Frame 2")
    plt.axis("off")

    plt.show()
else:
    print("Unable to read video frames.")
```



```
# Step 3: Implement Lucas-Kanade Optical Flow

cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

ret, old_frame = cap.read()

old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

ret, new_frame = cap.read()

new_gray = cv2.cvtColor(new_frame, cv2.COLOR_BGR2GRAY)

p1, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray,
    new_gray,
    p0,
    None
)

good_new = p1[status == 1]
good_old = p0[status == 1]

print("Tracked feature points:", len(good_new))
```

```
Tracked feature points: 55
```

```
# Step 4: Visualize Lucas-Kanade motion vectors

lucas_image = new_frame.copy()

for new, old in zip(good_new, good_old):
    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    cv2.arrowedLine(
        lucas_image,
        (int(x_old), int(y_old)),
        (int(x_new), int(y_new)),
        (0, 255, 0),
        2
    )

    cv2.circle(
        lucas_image,
        (int(x_new), int(y_new)),
        3,
        (0, 0, 255),
        -1
    )

plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(lucas_image, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Optical Flow")
```

```
plt.axis("off")
plt.show()
```




```
# Step 5: Implement Farneback Dense Optical Flow

cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

ret, frame1 = cap.read()
ret, frame2 = cap.read()

gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

print("Farneback Dense Optical Flow calculated.")
```

```
Farneback Dense Optical Flow calculated.
```

```
# Step 6: Display dense optical flow using color visualization

magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

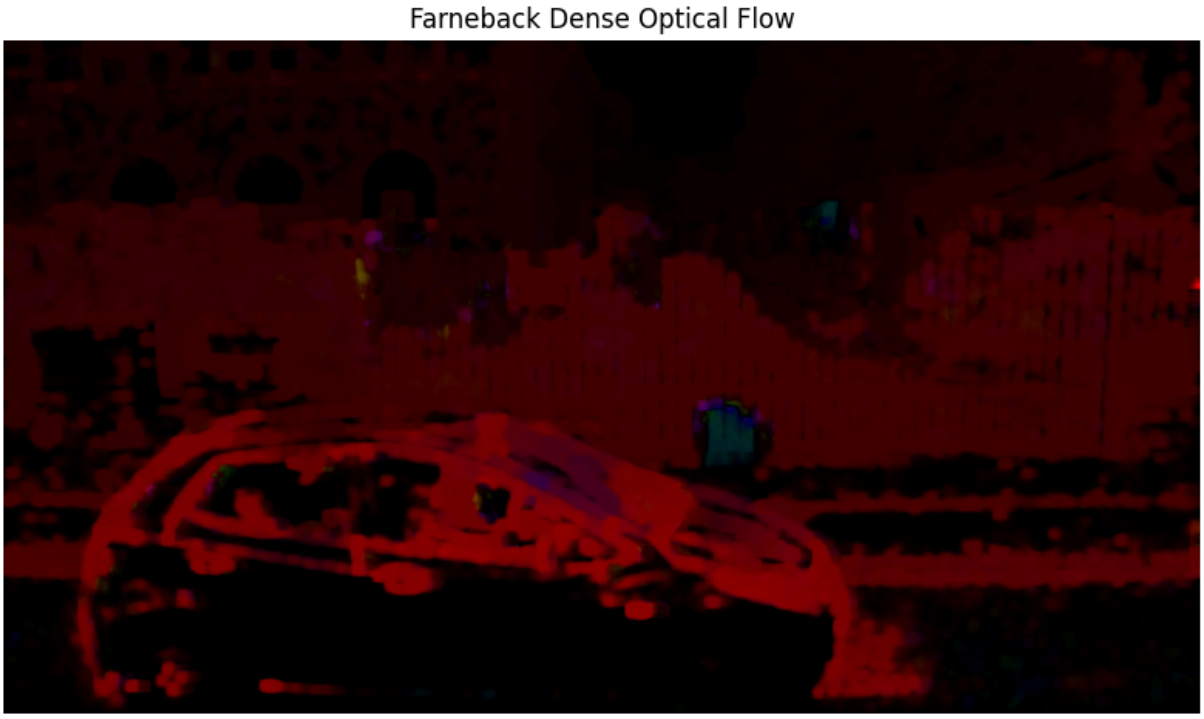
hsv = np.zeros_like(frame1)

hsv[..., 0] = angle * 180 / np.pi / 2
```

```
hsv[..., 1] = 255
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

dense_flow = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2RGB
)

plt.figure(figsize=(10, 6))
plt.imshow(dense_flow)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()
```




```
# Step 7: Compare Lucas-Kanade and Farneback algorithms

print("Lucas-Kanade Optical Flow:")
print("- Tracks selected feature points.")
print("- Computationally efficient.")
print("- Suitable for object tracking.")
print("- Does not calculate motion for every pixel.")

print("\nFarneback Dense Optical Flow:")
print("- Calculates motion for almost every pixel.")
print("- Provides detailed motion information.")
print("- More computationally expensive.")
print("- Useful for dense motion analysis.")
```

```
Lucas-Kanade Optical Flow:
- Tracks selected feature points.
- Computationally efficient.
- Suitable for object tracking.
- Does not calculate motion for every pixel.
```

```
Farneback Dense Optical Flow:
- Calculates motion for almost every pixel.
- Provides detailed motion information.
- More computationally expensive.
- Useful for dense motion analysis.
```

```
# Step 8: Analyze effects of speed, lighting and camera motion

print("Effect of object speed:")
print("Fast-moving objects can produce larger motion vectors and tracking errors.")

print("\nEffect of lighting:")
print("Sudden lighting changes can reduce the accuracy of optical flow.")

print("\nEffect of camera motion:")
print("Camera movement can create motion across a large part of the image.")

print("\nStable lighting and moderate object speed generally provide better results.")
```

```
Effect of object speed:
Fast-moving objects can produce larger motion vectors and tracking errors.

Effect of lighting:
Sudden lighting changes can reduce the accuracy of optical flow.

Effect of camera motion:
Camera movement can create motion across a large part of the image.

Stable lighting and moderate object speed generally provide better results.
```

```
# Step 9: Evaluate real-world applications

applications = {
    "Object Tracking": "Lucas-Kanade is useful for tracking selected objects or feature points.
    "Surveillance": "Farneback can provide detailed motion information across a scene.",
    "Autonomous Navigation": "Dense optical flow can help estimate scene motion.",
    "Video Analysis": "Both methods can be used to analyze movement between frames."
}

for application, use in applications.items():
    print(application + ":")
    print(use)
    print()
```

```
Object Tracking:
Lucas-Kanade is useful for tracking selected objects or feature points.

Surveillance:
Farneback can provide detailed motion information across a scene.

Autonomous Navigation:
Dense optical flow can help estimate scene motion.

Video Analysis:
Both methods can be used to analyze movement between frames.
```

```
# Step 10: Summarize observations

print("Observations:")
print("1. Optical flow estimates motion between consecutive video frames.")
print("2. Lucas-Kanade tracks selected feature points.")
print("3. Farneback estimates dense motion across the image.")
print("4. Lucas-Kanade generally requires less computation.")
print("5. Farneback provides more detailed motion information.")
print("6. Optical flow is important for tracking, surveillance, robotics,")
print("   autonomous navigation and dynamic scene analysis.")
```

```
cap.release()
```

```
Observations:
1. Optical flow estimates motion between consecutive video frames.
2. Lucas-Kanade tracks selected feature points.
3. Farneback estimates dense motion across the image.
4. Lucas-Kanade generally requires less computation.
5. Farneback provides more detailed motion information.
6. Optical flow is important for tracking, surveillance, robotics,
   autonomous navigation and dynamic scene analysis.
```